Customer Churn Prediction Using Naive Bayes (Custom Implementation):
Dataset used: Dataset

1. Business Understanding

Objective:

This work is aimed at implementing Naive Bayes from custom and then comparing the performance with that of **GaussianNB** in Scikit-learn. To perform the binary classification task, Customer Churn dataset used to predict whether a passenger is alive (1) or not alive (0).

Key Goals:

- Understanding how the Naive Bayes classification works.
- Investigate the importance of feature engineering and encoding.
- Evaluate and compare performance between custom and Scikit-learn implementations.
- Analyze how feature selection affects performance of the model.

2. Data Understanding

Each record in the dataset corresponds to a customer and contains the same set of features as the training file, such as age, gender, tenure, usage frequency, support calls, payment delay, subscription type, contract length, total spend, and last interaction. The dataset allows businesses to assess the predictive power of their trained models on unseen data and gain insights into how well the models generalize to new customers.

3. Data Preparation

This step follows:

- Handle missing values.
- Encode categorical features.
- Remove outliers by IQR technique
- Scale numerical features using **stdscaler**
- Split into train & test sets.

4. Modeling Implementations (custom)

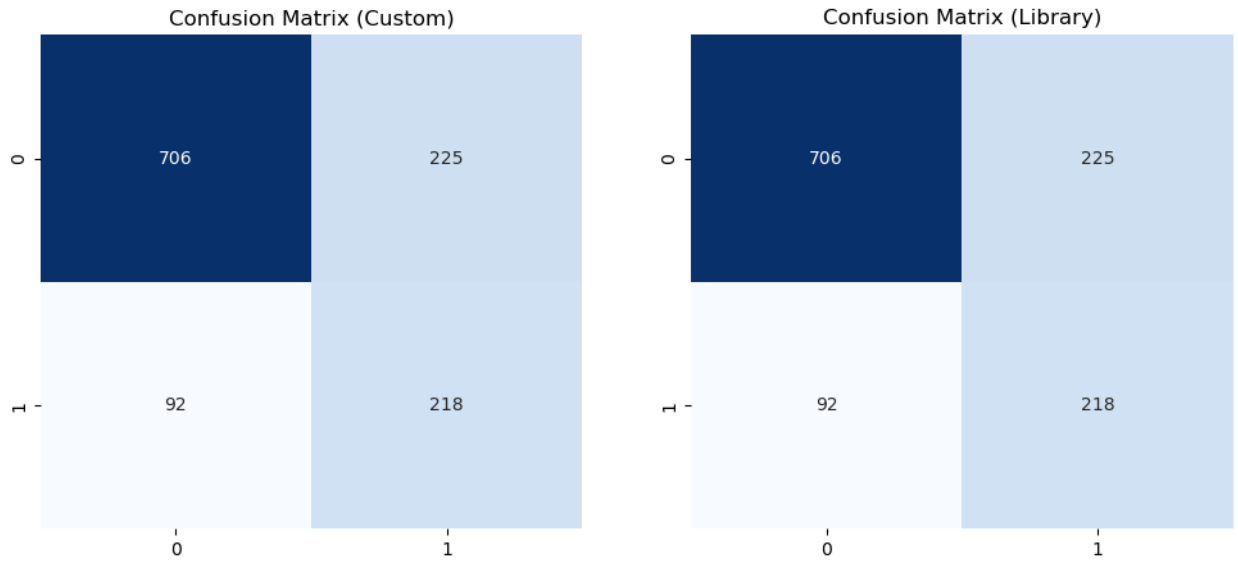
Implementation Details:

- The Naive Bayes Classifier assumes that inputs are independent features and proceeds to calculate probabilities according to: $P(Y \mid X) = P(X \mid Y)\, P(Y) / P(X)$
- Gaussian Naive Bayes assumes that the numerical feature will follow a normal distribution.
- Training for the model requires estimation of:
- Class Prior Probabilities (P(Y))

- Means and Variance of Features per class (P(X | Y))
- Prediction:
- These are posterior probabilities calculated using the Bayes' theorem for every class.
- The one that has maximum is picked.

5. Model Evaluation

```
Naive Bayes (custom) Accuracy: 0.7446
Naive Bayes (Library) Accuracy: 0.7446

Confusion Matrix (Custom):
 [[706 225]
 [ 92 218]]

Confusion Matrix (Library):
 [[706 225]
 [ 92 218]]

Classification Report (Custom):
              precision    recall  f1-score   support

           0       0.88      0.76      0.82       931
           1       0.49      0.70      0.58       310

    accuracy                           0.74      1241
   macro avg       0.69      0.73      0.70      1241
weighted avg       0.79      0.74      0.76      1241


Classification Report (Library):
              precision    recall  f1-score   support

           0       0.88      0.76      0.82       931
           1       0.49      0.70      0.58       310

    accuracy                           0.74      1241
   macro avg       0.69      0.73      0.70      1241
weighted avg       0.79      0.74      0.76      1241
```



From Observations:

- It achieves the exact same accuracy for custom implementations as for Scikit-learn implementations.
- Feature selection and feature encoding are very critical to achieving that good accuracy.
- Naive Bayes works wonders so far, at least for categorical data and this one happens to be fit in that manner.

6. Conclusion & Future Improvements

- The custom implementation of Naive Bayes has been able to replicate the result produced by the Scikit-learn implementation of **GaussianNB**.
- Feature engineering has had a large impact on the performance of the model.
- Naive Byes is a very strong baseline model for classification tasks, especially when the assumption of independence on features holds true.

Coding Implementation:

Function details

- **Fit**: Calculate means, variances, priors.
- **Predict**: Compute likelihood, priors, posterior.
- **Classes**: Identify unique class labels.
- **Likelihood**: Use Gaussian probability distribution.
- **Prediction**: Choose class with highest posterior

Flowchart

1. Start

   |

2. Fit Method

   |

   |-----> Calculate Unique Classes

   | |-----> For each class:

   | |-----> Calculate Mean

   | |-----> Calculate Variance

   | |-----> Calculate Prior Probability

   |

3. Predict Method

   |

   |-----> For each test sample (X):

   | |-----> For each class:

   | |-----> Calculate Likelihood using Gaussian formula

   | |-----> Multiply Likelihood by Prior Probability

   | |-----> Select class with highest posterior probability

   |

4. Output Prediction